

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium) Assessment

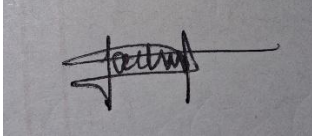
Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I Jashwant.G.R-192524384 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Online Shopping Platform – Normalize Order to 3NF

Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Assumptions

- OrderID uniquely identifies a single order.
- CustID uniquely identifies a customer.
- ProdID uniquely identifies a product.
- One order may contain several different products.
- A given product appears at most once within the same order.
- Price refers to the standard selling price of the product.

Step 1: Identify the Functional Dependencies

1. **OrderID $\rightarrow$ CustID**
2. **CustID $\rightarrow$ CustName**
3. **ProdID $\rightarrow$ ProdName, Price**
4. **(OrderID, ProdID) $\rightarrow$ Qty**

Since OrderID determines CustID and CustID determines CustName, $\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$ is a transitive dependency. Because an order can hold multiple products, the candidate key of the un-normalized relation is the composite (OrderID, ProdID).

Step 2: Check for Partial Dependencies (2NF)

With the candidate key (OrderID, ProdID):

- $\text{OrderID} \rightarrow \text{CustID}$ depends only on part of the key.
- $\text{ProdID} \rightarrow \text{ProdName, Price}$ also depends only on part of the key.

These are partial dependencies, so the relation violates 2NF.

Step 3: Decompose to Remove Partial Dependencies

ORDER_HEADER(OrderID, CustID)

PRODUCT(ProdID, ProdName, Price)

ORDER_ITEM(OrderID, ProdID, Qty)

Step 4: Remove the Transitive Dependency (3NF)

OrderID $\rightarrow$ CustID $\rightarrow$ CustName is transitive inside ORDER_HEADER, so CustName is separated out:

CUSTOMER(CustID, CustName)

Final 3NF Schema

- CUSTOMER(CustID, CustName) — Primary Key: CustID
- PRODUCT(ProdID, ProdName, Price) — Primary Key: ProdID
- ORDER_HEADER(OrderID, CustID) — Primary Key: OrderID, Foreign Key: CustID
- ORDER_ITEM(OrderID, ProdID, Qty) — Primary Key: (OrderID, ProdID), Foreign Keys: OrderID, ProdID

Conclusion

All partial and transitive dependencies have been eliminated. Customer, product, order-header, and order-item facts are now stored in separate relations, which removes redundancy and prevents insertion, update, and deletion anomalies. The design is in 3NF.

2. Hospital Database – Normalize Patient to BCNF

Given Relation

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assumptions

- PID uniquely identifies a patient.
- DocID uniquely identifies a doctor.
- WardNo uniquely identifies a ward.
- Each patient is currently attached to exactly one doctor and one ward.

Step 1: Functional Dependencies

5. **PID $\rightarrow$ PName, DocID, WardNo**
6. **DocID $\rightarrow$ DocName, Specialization**
7. **WardNo $\rightarrow$ WardName**

PID $\rightarrow$ DocID $\rightarrow$ DocName, Specialization and PID $\rightarrow$ WardNo $\rightarrow$ WardName are both transitive dependencies.

Step 2: Candidate Key

Closure of PID: $\{PID\} \rightarrow \{PID, PName, DocID, WardNo\} \rightarrow \{PID, PName, DocID, DocName, Specialization, WardNo, WardName\}$. Since the closure covers every attribute, Candidate Key = PID.

Step 3: Anomalies in the Un-normalized Table

- Update anomaly: a change in a doctor's specialization must be repeated in every patient row linked to that doctor.
- Insertion anomaly: a newly hired doctor cannot be recorded until at least one patient is assigned to them.
- Deletion anomaly: discharging the only patient under a doctor erases all record of that doctor.

Step 4: BCNF Decomposition

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

Step 5: Verify BCNF

In PATIENT, PID is the key; in DOCTOR, DocID is the key; in WARD, WardNo is the key. Every determinant in every resulting relation is a superkey, so all three relations satisfy BCNF.

Conclusion

Separating doctor and ward details from the patient record removes the transitive dependencies and the associated update, insertion, and deletion anomalies. The final design PATIENT / DOCTOR / WARD is in BCNF.

3. University Enrollment – Analyze FDs and Apply 2NF

Given Relation

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Assumptions

- SID uniquely identifies a student; CourseID uniquely identifies a course.
- A student may enroll in several courses, and a course may have many students.
- Grade is specific to a particular student in a particular course.

Step 1: Functional Dependencies

8. $SID \rightarrow SName$
9. $CourseID \rightarrow CourseName, Faculty$
10. $(SID, CourseID) \rightarrow Grade$

Step 2: Candidate Key

Closure of $(SID, CourseID) = \{SID, CourseID, SName, CourseName, Faculty, Grade\}$, i.e. all attributes.

Candidate Key = $(SID, CourseID)$.

Step 3: Partial Dependencies

$SName$ depends only on SID , and $CourseName/Faculty$ depend only on $CourseID$ — both are partial dependencies on the composite key, so the relation is not in 2NF.

Step 4: Decompose to 2NF

- $STUDENT(SID, SName)$ — Primary Key: SID
- $COURSE(CourseID, CourseName, Faculty)$ — Primary Key: $CourseID$
- $ENROLLMENT(SID, CourseID, Grade)$ — Primary Key: $(SID, CourseID)$

Conclusion

Student details now depend only on SID , course details depend only on $CourseID$, and Grade depends on the full composite key. All partial dependencies are removed, so $STUDENT$, $COURSE$, and $ENROLLMENT$ satisfy 2NF.

4. Airline Booking – Normalize to BCNF

Given Relation

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assumptions

- PassID uniquely identifies a passenger; FlightNo identifies a fixed flight route.
- A flight (FlightNo) operates on many different dates.
- A seat number is unique only within a specific flight on a specific date.

Step 1: Functional Dependencies

- 11. **PassID** $\rightarrow$ **PassName**
- 12. **FlightNo** $\rightarrow$ **DeptCity, ArrCity**
- 13. **(FlightNo, Date, PassID)** $\rightarrow$ **Seat**
- 14. **(FlightNo, Date, Seat)** $\rightarrow$ **PassID**

Step 2: Candidate Keys

Both (FlightNo, Date, PassID) and (FlightNo, Date, Seat) determine every attribute of the relation, so both are candidate keys.

Step 3: BCNF Violations

PassID $\rightarrow$ PassName violates BCNF because PassID alone is not a superkey. Likewise, FlightNo $\rightarrow$ DeptCity, ArrCity violates BCNF because FlightNo alone is not a superkey (the same flight number recurs across many dates and passengers).

Step 4: BCNF Decomposition

PASSENGER(PassID, PassName)

FLIGHT(FlightNo, DeptCity, ArrCity)

BOOKING(FlightNo, Date, PassID, Seat)

Step 5: Verify BCNF

PassID is the key of PASSENGER, FlightNo is the key of FLIGHT, and in BOOKING both (FlightNo, Date, PassID) and (FlightNo, Date, Seat) are candidate keys. Every determinant is now a superkey.

Conclusion

Separating passenger and flight-route information from the individual booking record removes both BCNF violations. PASSENGER, FLIGHT, and BOOKING together form a lossless BCNF design.

5. HR System – Functional and Transitive Dependencies, 3NF

The question does not give exact attributes, so a representative multinational HR schema is used:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, CountryID, CountryName)

Assumptions

- EmpID uniquely identifies an employee; DeptID uniquely identifies a department; CountryID uniquely identifies a country.
- Each department has one department name and one manager.
- Each country has a single country name.

Step 1: Functional Dependencies

15. **EmpID** $\rightarrow$ **EmpName, DeptID, ManagerID, CountryID**

16. **DeptID** $\rightarrow$ **DeptName, ManagerID**

17. **CountryID** $\rightarrow$ **CountryName**

Step 2: Transitive Dependencies

EmpID $\rightarrow$ DeptID and DeptID $\rightarrow$ DeptName, ManagerID together give EmpID $\rightarrow$ DeptName, ManagerID (transitive). Similarly, EmpID $\rightarrow$ CountryID and CountryID $\rightarrow$ CountryName give EmpID $\rightarrow$ CountryName (transitive).

Step 3: Candidate Key

EmpID+ covers every attribute in the relation, so Candidate Key = EmpID.

Step 4: 3NF Decomposition

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, CountryID)

DEPARTMENT(DeptID, DeptName, ManagerID)

COUNTRY(CountryID, CountryName)

Step 5: Verify 3NF

EmpID is the key of EMPLOYEE, DeptID is the key of DEPARTMENT, and CountryID is the key of COUNTRY — no remaining non-key attribute depends transitively on the primary key of its relation.

Conclusion

Separating department and country information removes both transitive dependencies. The resulting EMPLOYEE, DEPARTMENT, and COUNTRY relations are in 3NF, reducing redundancy and improving consistency across the multinational HR system.

6. Library – Identify MVDs and Decompose to 4NF

Given Relation

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Assumption

A member may borrow many books, and the set of books linked to a member is treated as an independent multivalued fact; each book's own details are fixed by BookID.

Step 1: Functional Dependencies

18. **MemberID** → **MemberName**

19. **BookID** → **Title, Author, Genre**

20. **(MemberID, BookID)** → **BorrowDate**

Step 2: Multivalued Dependency

Because the collection of books associated with a member is independent of any other multivalued fact, $\text{MemberID} \twoheadrightarrow \text{BookID}$ holds. Storing this multivalued fact together with member and book attributes in one relation causes repeated combinations, so the relation is not in 4NF even though it may already be in BCNF.

Step 3: Redundancy Caused by the MVD

MemberName is repeated once for every book a member has borrowed, and book details are repeated whenever the same book is borrowed by different members.

Step 4: 4NF Decomposition

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, Author, Genre)

BORROWING(MemberID, BookID, BorrowDate)

Step 5: Verify 4NF

MemberID is the key of MEMBER, BookID is the key of BOOK, and the composite (MemberID, BookID) is the key of BORROWING. No non-trivial multivalued dependency remains inside any of the three relations.

Conclusion

Splitting member facts, book facts, and the borrowing event into separate relations removes the multivalued redundancy, giving a 4NF design under the stated independence assumption.

7. Telecom Billing – Candidate Keys, Primary Key, and FDs

The question does not specify exact attributes, so a representative telecom billing schema is used:

BILLING(MobileNo, CustomerName, PlanID, PlanName, CallID, CallDate, Duration, Rate, Amount)

Assumptions

- MobileNo uniquely identifies a customer; PlanID uniquely identifies a plan; CallID uniquely identifies a call record.
- Every call belongs to exactly one mobile number.
- The rate applied to a call is determined by the customer's plan.

Step 1: Functional Dependencies

21. **MobileNo** $\rightarrow$ **CustomerName, PlanID**

22. **PlanID** $\rightarrow$ **PlanName, Rate**

23. **CallID** $\rightarrow$ **MobileNo, CallDate, Duration**

24. **CallID** $\rightarrow$ **Amount**

Since $\text{CallID} \rightarrow \text{MobileNo}$ and $\text{MobileNo} \rightarrow \text{CustomerName, PlanID}$, and $\text{PlanID} \rightarrow \text{PlanName, Rate}$, the chain $\text{CallID} \rightarrow \text{MobileNo} \rightarrow \text{CustomerName, PlanID} \rightarrow \text{PlanName, Rate}$ holds transitively.

Step 2: Determine the Candidate Key

Closure of CallID: $\{\text{CallID}\} \rightarrow \{\text{CallID, MobileNo, CallDate, Duration}\} \rightarrow \{\dots, \text{CustomerName, PlanID}\} \rightarrow \{\dots, \text{PlanName, Rate}\}$. The closure reaches every attribute, so Candidate Key = CallID.

Step 3: Primary Key

Primary Key = CallID, since it is the single candidate key that uniquely identifies each billing record.

Step 4: Non-Prime Attributes

All attributes other than CallID (MobileNo, CustomerName, PlanID, PlanName, CallDate, Duration, Rate, Amount) are non-prime.

Final Answer

- Candidate Key: CallID
- Primary Key: CallID
- Key Functional Dependencies: $\text{MobileNo} \rightarrow \text{CustomerName, PlanID}$; $\text{PlanID} \rightarrow \text{PlanName, Rate}$; $\text{CallID} \rightarrow \text{MobileNo, CallDate, Duration}$; $\text{CallID} \rightarrow \text{Amount}$

Conclusion

The transitive chain from CallID through MobileNo and PlanID shows that customer, plan, and call details should be split into separate relations during normalization to avoid redundancy and update anomalies.

8. Retail Inventory – Complete Normalization

Given Relation

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assumptions

- PID uniquely identifies a product; SupplierID uniquely identifies a supplier.
- A product can be stocked in several warehouses, and a warehouse can hold many products.
- Price is fixed per product regardless of warehouse.

Step 1: Functional Dependencies

25. **PID $\rightarrow$ PName, SupplierID, Category, Price**

26. **SupplierID $\rightarrow$ SupplierName**

PID $\rightarrow$ SupplierID $\rightarrow$ SupplierName is a transitive dependency. Because a product can exist in multiple warehouses, there is also a many-to-many relationship between PID and Warehouse.

Step 2: Candidate Key

Since a product may appear in more than one warehouse, the key of the un-normalized relation is the composite (PID, Warehouse).

Step 3: Partial Dependencies

PID $\rightarrow$ PName, SupplierID, Category, Price depends only on PID, which is part of the composite key — a partial dependency, so the relation violates 2NF.

Step 4: Decompose to 2NF

PRODUCT(PID, PName, SupplierID, Category, Price)

INVENTORY(PID, Warehouse)

Step 5: Remove the Transitive Dependency (3NF/BCNF)

PID $\rightarrow$ SupplierID and SupplierID $\rightarrow$ SupplierName form a transitive chain inside PRODUCT, so supplier details are separated out:

SUPPLIER(SupplierID, SupplierName)

Final Normalized Schema

- **PRODUCT(PID, PName, SupplierID, Category, Price)** — Primary Key: PID
- **SUPPLIER(SupplierID, SupplierName)** — Primary Key: SupplierID
- **INVENTORY(PID, Warehouse)** — Primary Key: (PID, Warehouse)

Normalization Verification

- **1NF**: every attribute holds a single atomic value.
- **2NF**: the partial dependency on PID has been removed by separating PRODUCT from INVENTORY.
- **3NF**: the transitive dependency $\text{SupplierID} \rightarrow \text{SupplierName}$ has been removed by separating SUPPLIER.
- **BCNF**: in every resulting relation, the determinant of each dependency is a key.

Conclusion

The retail inventory schema is fully normalized up to BCNF under the stated assumptions, with product, supplier, and multi-warehouse stock information stored independently to eliminate redundancy.

9. School ERP – Normalize up to BCNF

As no specific schema is provided, a representative poorly designed School ERP relation is used:

SCHOOL_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

Assumptions

- StudentID, ClassID, TeacherID, and SubjectID are each unique identifiers.
- A student studies multiple subjects and belongs to one class.
- Grade is determined jointly by the student and the subject.

Step 1: Functional Dependencies

27. **$\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$**

28. **$\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$**

29. **$\text{TeacherID} \rightarrow \text{TeacherName}$**

30. **$\text{SubjectID} \rightarrow \text{SubjectName}$**

31. **$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{Grade}$**

Step 2: Candidate Key

(StudentID, SubjectID) determines every attribute in the relation, so Candidate Key = (StudentID, SubjectID).

Step 3: Partial Dependencies

StudentID alone determines StudentName and ClassID, and SubjectID alone determines SubjectName — both are partial dependencies, so the relation violates 2NF.

Step 4: Decompose to 2NF

STUDENT(StudentID, StudentName, ClassID)

SUBJECT(SubjectID, SubjectName)

RESULT(StudentID, SubjectID, Grade)

Step 5: Remove Transitive Dependencies

Inside STUDENT, ClassID → ClassName, TeacherID and TeacherID → TeacherName form a transitive chain, so class and teacher details are separated:

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

Final BCNF Schema

- STUDENT(StudentID, StudentName, ClassID)
- CLASS(ClassID, ClassName, TeacherID)
- TEACHER(TeacherID, TeacherName)
- SUBJECT(SubjectID, SubjectName)
- RESULT(StudentID, SubjectID, Grade)

BCNF Verification

StudentID is the key of STUDENT, ClassID is the key of CLASS, TeacherID is the key of TEACHER, SubjectID is the key of SUBJECT, and the full composite (StudentID, SubjectID) is the key of RESULT. Every determinant is a superkey in its own relation.

Conclusion

The School ERP schema is normalized up to BCNF. Removing the partial and transitive dependencies eliminates redundancy and prevents insertion, update, and deletion anomalies across student, class, teacher, and subject records.

10. Movie Streaming Platform – Join Dependencies and 5NF

As the exact schema is not specified, a representative relation is used:

MOVIE_STREAM(MovieID, ActorID, PlatformID)

Assumption

A movie can feature multiple actors and can be available on multiple platforms, and an actor can appear on multiple platforms. These three pairwise relationships are independent of one another, so the three-way relation is subject to a non-trivial join dependency rather than a simple multivalued dependency.

Step 1: Identify the Join Dependency

The relation combines three independent pairwise facts — Movie–Actor, Movie–Platform, and Actor–Platform — so:

MOVIE_STREAM = MOVIE_ACTOR $\bowtie$ MOVIE_PLATFORM $\bowtie$ ACTOR_PLATFORM

Step 2: Redundancy in the Un-decomposed Relation

If a movie has several actors and is offered on several platforms, every valid movie–actor–platform combination must be stored, causing the same movie–actor and movie–platform facts to repeat many times.

Step 3: Decompose to 5NF

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Step 4: Lossless-Join Verification (Example)

Suppose MOVIE_ACTOR contains (M1, A1) and (M1, A2); MOVIE_PLATFORM contains (M1, P1) and (M1, P2); and ACTOR_PLATFORM contains (A1, P1) and (A2, P2). Joining the three relations on their common attributes reconstructs exactly the valid (MovieID, ActorID, PlatformID) combinations permitted by the business rules, with no spurious tuples introduced.

Step 5: 5NF Verification

A relation is in 5NF (Project–Join Normal Form) when every non-trivial join dependency it satisfies is implied by its candidate keys, so no further lossless decomposition is possible. Because MOVIE_ACTOR, MOVIE_PLATFORM, and ACTOR_PLATFORM each represent one independent pairwise relationship, none of them can be decomposed further without losing information.

Conclusion

Decomposing MOVIE_STREAM into MOVIE_ACTOR, MOVIE_PLATFORM, and ACTOR_PLATFORM removes the redundancy caused by the join dependency. The decomposition is lossless under the stated independence assumption, and the resulting design is in 5NF.